

University of Science and Technology of Hanoi



MACHINE LEARNING IN MEDICINE

Final Report

Brain hemorrhage segmentation

Group Members:

Dinh Thanh Hai	22BA13119	haidt.22ba13119@usth.edu.vn
Tran Huy Quan	22BA13260	quanth.22ba13260@usth.edu.vn
Ta Quang Dung	22BA13089	dungtq.22ba13089@usth.edu.vn
Nguyen Minh Duc	22BA13080	ducnm.22ba13080@usth.edu.vn
Nguyen Truong Giang	23BI14139	giangnt.23bi14139@usth.edu.vn
Do Hong Chung	22BA13055	chungdh.22ba13055@usth.edu.vn

Hanoi, March 2026

Contents

1	INTRODUCTION	3
2	DATASET AND PREPROCESSING	5
3	METHODOLOGY	9
3.1	ResNet50 Encoder	9
3.2	SegResNet	11
4	EXPERIMENT AND RESULTS	14
4.1	ResNet50 Encoder	14
4.2	SegResNet	17
4.3	Discussion	20
5	CONCLUSION	22

List of Figures

1.1	Overall pipeline	3
2.1	Brain scan slice and corresponding hemorrhage segmentation	5
2.2	Visualization of a 3D brain scan stored in NIfTI format	6
2.3	Overview of the preprocessing pipeline used before training the segmentation model.	7
4.1	Training curves. Left: train and validation loss. Center: validation metrics over epochs. Right: learning rate schedule (warmup + cosine decay) . . .	14
4.2	Box plots of test metrics (left) and HD95 distance (right) across 50 test volumes.	15
4.3	Prediction overlay on a single slice	16
4.4	3D mesh of the best case (cerebral_185, Dice = 0.917)	17
4.5	Training curves	18
4.6	Distribution of segmentation metrics and HD95 on the test set.	19
4.7	Visual comparison on test case cerebral_161_0_0.nii (z=4)	19
4.8	3D hemorrhage mesh on cerebral_189_0_0.nii.	20

Chapter 1

INTRODUCTION

Overall pipeline:

The overall workflow of the proposed system is illustrated in Figure 1.1. The process begins with loading brain CT scans stored in NIfTI format. After preprocessing steps such as normalization and resizing, the dataset is divided into training, validation, and testing subsets. The preprocessed volumes are then fed into a 3D U-Net segmentation model. During training, the model learns to predict hemorrhage regions by optimizing a segmentation loss function. Finally, the trained model is evaluated using several metrics including Dice score, IoU, precision, and recall, and the segmentation results are visualized for qualitative analysis.

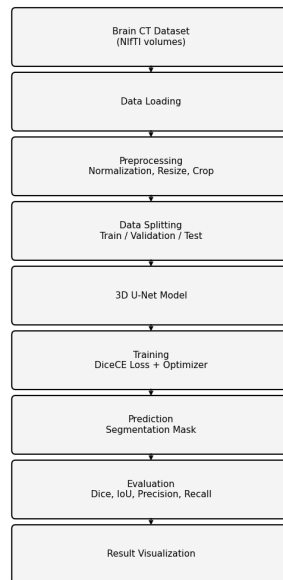


Figure 1.1: Overall pipeline

Project Context

Brain hemorrhage is a dangerous medical condition that occurs when there is bleeding inside the brain. This condition can cause serious damage to brain tissue and in severe cases may lead to death if it is not detected early. Because of this, early diagnosis plays an important role in medical treatment. Doctors usually analyze brain imaging data such as CT or MRI scans to locate the bleeding regions and evaluate their severity.

However, examining medical images manually can be difficult and time-consuming, especially when the number of scans is large. The process also depends on the experience of the radiologist, which may lead to differences in interpretation. In recent years, machine learning and deep learning methods have been increasingly applied in medical imaging to help automate some of these tasks. One important task in medical image analysis is segmentation, where the goal is to identify specific regions inside an image. In the case of brain hemorrhage, segmentation helps highlight the exact area of bleeding in the brain scan. Accurate segmentation can support doctors in understanding the extent of the hemorrhage and assist with treatment planning.

In this project, we focus on the problem of brain hemorrhage segmentation using 3D medical imaging data. The dataset used in this work consists of brain scans stored in the NIfTI (.nii) format, which is commonly used in medical imaging research. Each scan is accompanied by a corresponding mask that indicates the hemorrhage region. Because the scans are stored as 3D volumes, the model needs to process spatial information across multiple slices. To address this problem, a deep learning segmentation model based on the 3D U-Net architecture is implemented using the MONAI framework and PyTorch.

Research Objectives

The main goal of this project is to develop a deep learning model that can automatically segment brain hemorrhage regions from medical images. More specifically, the project aims to train a 3D segmentation model that can learn patterns from brain scan data and predict the location of hemorrhage regions. The dataset consists of 3D brain images and their corresponding segmentation masks stored in NIfTI format.

Another objective is to evaluate how well the model performs on this task. Several evaluation metrics such as Dice score, IoU, precision, and recall are used to measure the segmentation performance. These metrics help determine whether the predicted regions are similar to the ground truth annotations.

In addition, visualization of the predicted results is used to compare the model output with the ground truth segmentation. This helps provide a clearer understanding of how accurately the model can detect hemorrhage areas in the brain scans.

Chapter 2

DATASET AND PREPROCESSING

Dataset Description

The dataset used in this project is a brain hemorrhage segmentation dataset that contains medical images stored in the NIfTI (.nii) format. This format is widely used in medical imaging because it can store three-dimensional volumetric data along with spatial information. Each sample in the dataset consists of a 3D brain scan and a corresponding segmentation mask. The brain scan represents the original medical image, while the mask indicates the location of the hemorrhage region. In the mask file, the hemorrhage area is labeled so that it can be used as the ground truth during model training. This allows the segmentation model to learn how to distinguish between normal brain tissue and hemorrhage regions.

Each scan is paired with a corresponding segmentation mask that indicates the hemorrhage region. The mask highlights the location of bleeding inside the brain scan and serves as the ground truth for training the segmentation model.

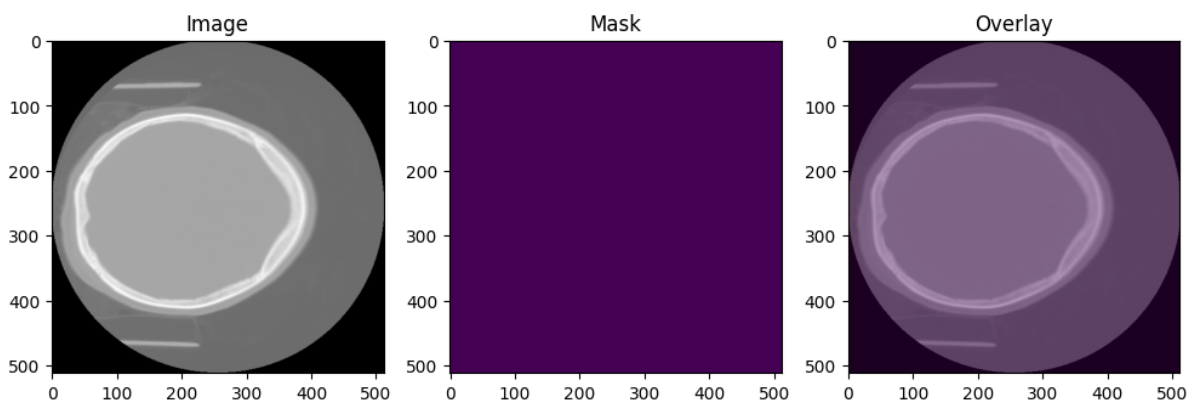


Figure 2.1: Brain scan slice and corresponding hemorrhage segmentation

Figure 2.1 shows an example of a brain slice together with its corresponding hemorrhage mask.

Unlike regular image datasets where images are stored as 2D pictures, medical imaging datasets often contain full 3D volumes. Each volume is composed of many slices that together form a complete representation of the brain. Because hemorrhage can appear in different parts of the brain, using the full 3D structure can help the model learn spatial relationships between slices. The dataset is organized into pairs of image files and label files. Each image volume has a corresponding mask volume with the same shape. This pairing is important because the model needs both the image and the ground truth segmentation during training.

Data Format and Structure

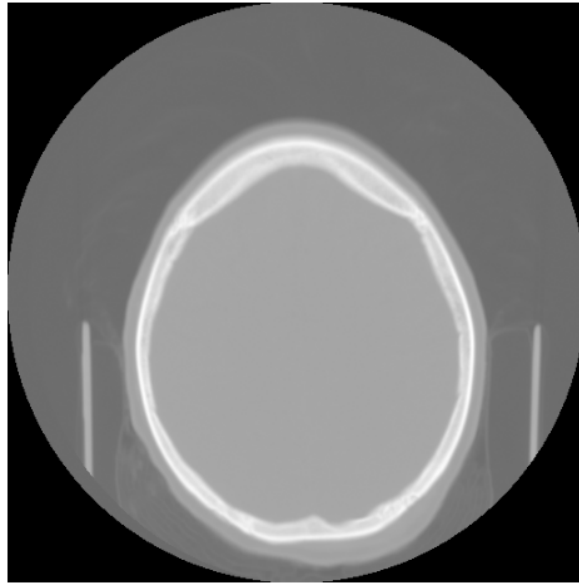


Figure 2.2: Visualization of a 3D brain scan stored in NIfTI format

All brain scans in the dataset are stored in the NIfTI format (.nii or .nii.gz). This format is commonly used in neuroimaging research and can store medical images in three dimensions. Each NIfTI file contains a volumetric representation of the brain. The data can be interpreted as a 3D matrix where each voxel represents the intensity value at a specific position in the brain. In the segmentation masks, the voxel values represent whether that location belongs to the hemorrhage region or not.

Working with 3D data introduces additional complexity compared to standard image datasets. The model must process spatial information along three axes instead of just two. However, this also allows the model to capture more contextual information from the surrounding slices.

Data Preprocessing

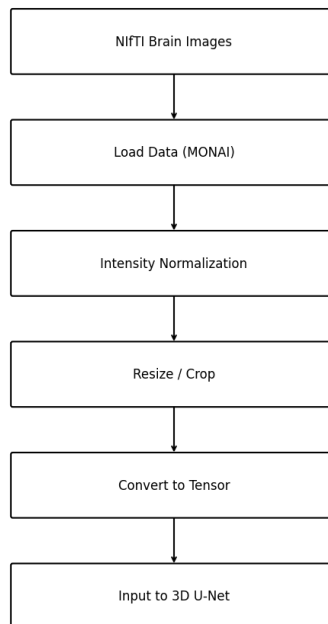


Figure 2.3: Overview of the preprocessing pipeline used before training the segmentation model.

Before training the segmentation model, several preprocessing steps are applied to prepare the dataset. These steps are necessary to ensure that the input data is consistent and suitable for deep learning models. First, the NIFTI files are loaded using medical imaging libraries that support volumetric data. After loading the scans, the image volumes and their corresponding masks are paired together so that they can be used during the training process. Next, the images are normalized to make the intensity values more consistent across different scans. Normalization helps improve model training because neural networks tend to perform better when the input values are within a similar range.

In addition, the spatial dimensions of the images may be adjusted so that all samples have a consistent size. This is important because deep learning models usually require fixed input dimensions. Resizing or cropping techniques can be used to ensure that the data fits the required input size. Another important step is converting the data into tensors so that it can be processed by the deep learning framework. In this project, the PyTorch framework is used, and the preprocessing pipeline is implemented with the MONAI library, which provides many tools designed specifically for medical imaging.

Data Augmentation

Medical imaging datasets are often limited in size, which can make it difficult for deep learning models to generalize well. To address this issue, data augmentation techniques can be applied during training. Data augmentation involves creating modified versions of the original images while preserving their important features. For example, transfor-

mations such as flipping, rotation, or slight spatial changes can be applied to the input images and their corresponding masks. These transformations help increase the diversity of the training data and reduce the risk of overfitting. As a result, the model can learn more robust features and perform better when tested on unseen data.

Data Splitting

To evaluate the performance of the segmentation model, the dataset is divided into different subsets. Typically, the data is split into a training set and a validation or test set. The training set is used to train the model and learn the parameters of the neural network. The validation or test set is used to evaluate how well the model performs on data that it has not seen before. This separation helps provide a more reliable measurement of the model's ability to generalize to new brain scans. The evaluation results obtained from the test set are later used to analyze the effectiveness of the proposed segmentation approach.

Chapter 3

METHODOLOGY

3.1 ResNet50 Encoder

Model Architecture

We used a U-Net model with a ResNet50 encoder from the Segmentation Models PyTorch (SMP) library. The ResNet50 encoder was pretrained on ImageNet, which means it already learned useful features from natural images. The U-Net decoder has skip connections that help the model produce detailed segmentation maps by combining low-level and high-level features.

The model takes a 3-channel image of size 512×512 as input and outputs a single-channel map. We froze only the very first layer of the encoder (conv1 and bn1) and let all other layers (layer1 through layer4) be fine-tuned during training. We made this choice because the first layer captures basic edges and textures that are useful for any image, while the deeper layers need to adapt to the medical domain. In total, the model has 32,521,105 parameters, and 32,511,569 of them (99.97%) are trainable.

We also added a Dropout2D layer ($p=0.3$) before the final output layer. This randomly turns off some spatial features during training, which helps prevent overfitting. The decoder uses batch normalization for more stable training.

Loss Function

We combined two loss functions for training:

$$\mathcal{L} = \mathcal{L}_{\text{Dice}} + \mathcal{L}_{\text{Focal}} \quad (3.1)$$

We used Dice Loss because it directly measures the overlap between the prediction and the ground truth. It works well when the classes are very imbalanced, which is our case - hemorrhage usually takes up less than 1% of the total volume.

We also used Focal Loss ($\alpha = 0.25$, $\gamma = 2.0$) because it gives more attention to hard-to-classify pixels near the edges of the hemorrhage. Together, these two losses provide strong learning signals for both large and small hemorrhage regions.

Training Strategy

Parameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
LR schedule	Warmup (3 epochs) + Cosine decay
Warmup range	$1 \times 10^{-6} \rightarrow 1 \times 10^{-4}$
Minimum LR (cosine)	1×10^{-6}
Max epochs	50
Early stopping patience	8 epochs (based on val Dice)
Batch size	4
Mixed precision (AMP)	Enable (FP16)
Gradient clipping	Max norm = 1.0
torch.compile	Yes (PyTorch 2.0+)
GPU	NVIDIA Tesla T4 (14.6 GB VRAM)

Table 3.1: Training hyperparameters.

For the learning rate schedule, we used a warmup phase for the first 3 epochs. During this phase, the learning rate slowly increased from a very small value (1% of the target) to the full value. This helps the model not to destroy the pretrained weights at the beginning. After the warmup, the learning rate followed a cosine curve and gradually decreased to $1e-6$. We also used early stopping: if the validation Dice did not improve for 8 epochs in a row, we stopped training and used the best checkpoint.

Inference Pipeline

When we tested the model on new volumes, we processed each volume slice by slice. For each slice, we resized it to 512×512 , copied it to 3 channels, and passed it through the model. The model gives a probability map, and we used a threshold of 0.5 to create a binary mask. We then resized the predicted mask back to the original size and stacked all slices together to get the full 3D prediction. We saved each prediction as a NIfTI file with the same header information as the original volume.

Evaluation Metrics

We evaluated the model using the following metrics on the full 3D volumes:

Dice coefficient: This measures how much the prediction overlaps with the ground truth. A score of 1 means perfect overlap, and 0 means no overlap at all. We used Dice as our main metric.

IoU (Intersection over Union): Similar to Dice but stricter. It divides the overlapping area by the total combined area of both prediction and ground truth.

Precision: Out of all the voxels that the model predicted as hemorrhage, how many are actually hemorrhage. High precision means few false alarms.

Recall: Out of all the actual hemorrhage voxels, how many did the model find. High recall means the model does not miss much.

HD95 (Hausdorff Distance 95th percentile): This measures the distance in millimeters between the predicted boundary and the ground truth boundary. We use the 95th percentile to ignore small outliers. A lower value means the boundaries are closer.

3.2 SegResNet

Model Architecture

In this study, we use the SegResNet model from the MONAI library for brain hemorrhage segmentation. SegResNet is a 3D encoder-decoder network that uses residual blocks. We choose this model because it works well with 3D medical images and it is not too heavy for training. Table 3.1 shows the configuration of the model that we use in this study.

Parameter	Value
spatial_dims	3
in_channels	1
out_channels	2
init_filters	32
blocks_down	(1, 2, 2, 4)
blocks_up	(1, 1, 1)
dropout_prob	0.2

Table 3.2: SegResNet configuration.

In this model, the encoder has 4 levels with 1, 2, 2, and 4 residual blocks. The number of filters starts from 32 and doubles at each level (32, 64, 128, 256). The decoder has 3 levels, and each level has 1 residual block. We use skip connections to connect the encoder and decoder at each level. Each residual block contains two 3D convolution layers with Group Normalization and ReLU activation. We set the input as 1 channel because CT images are grayscale, and the output is 2 channels for binary segmentation (background and hemorrhage). We also use dropout with probability 0.2 to reduce overfitting.

Loss Function

We use **DiceFocalLoss** from the MONAI library as the loss function. This loss combines two parts: Dice Loss and Focal Loss. We choose this loss because in our data, the hemorrhage region is very small compared to the whole brain volume, so the classes are highly imbalanced.

We set the parameters as follows: `to_onehot_y=True` to convert the label to one-hot format, `softmax=True` to apply softmax on the output, `gamma=2.0` for the Focal Loss to focus more on hard samples, and `lambda_dice=1.0`, `lambda_focal=1.0` to give equal weight to both losses.

Training Strategy

We train the model with the following hyperparameters:

Parameter	Value
Optimizer	AdamW ($lr = 3 \times 10^{-4}$, $weight_decay = 10^{-5}$)
LR Schedule	Linear Warmup (5 epochs) + Cosine Decay
Max epochs	50
Batch size	1
Patch size	$128 \times 128 \times 16$
Pos:Neg ratio	4 : 1
Num samples per volume	4
Early stopping	patience = 15 epochs (based on Dice)
Mixed Precision	AMP with GradScaler (FP16)
Cache rate	0.3

Table 3.3: Training hyperparameters.

We use the AdamW optimizer with a learning rate of 3×10^{-4} and weight decay of 10^{-5} . For the learning rate schedule, we apply a linear warmup for the first 5 epochs, after which the learning rate decreases following a cosine curve until it approaches zero. This schedule is implemented using the `LambdaLR` scheduler provided by PyTorch. We also use Automatic Mixed Precision (AMP) together with `GradScaler` to accelerate training and reduce GPU memory usage. During training, the forward pass runs in FP16 precision, while the gradient updates are performed in FP32 precision to maintain numerical stability.

For early stopping, the patience is set to 15 epochs. This means that if the Dice score on the validation set does not improve for 15 consecutive epochs, the training process is stopped. The model with the best validation Dice score is saved during training.

Data Preprocessing and Augmentation

We build two transform pipelines using MONAI: one for training and one for validation.

For training, we apply the following transforms in order. First, `LoadImaged` loads the NIfTI files. Then `EnsureChannelFirstd` adds the channel dimension. `Orientationd` is used to reorient the images to the RAS coordinate system.

Next, `ScaleIntensityRangePercentilesd` normalizes the intensity values using the 0.5 and 99.5 percentiles, mapping the output range to $[0, 1]$ with clipping. The transform `Lambdad` converts the label to a binary mask, where any value greater than 0 is set to 1. After that, `CropForegroundd` removes empty background regions. To generate training samples, we use `RandCropByPosNegLabeld` to randomly crop patches of size $128 \times 128 \times 16$ with a positive-to-negative ratio of 4 : 1 and 4 samples per volume. We then apply several random data augmentation techniques, including `RandFlipd` on all three axes with probability 0.5, `RandRotate90d` with probability 0.3, `RandGaussianNoised` with

probability 0.15 and $\sigma = 0.05$, and `RandGaussianSmoothd` with probability 0.1. Finally, `EnsureTyped` converts the data into PyTorch tensors.

For validation, we apply a simpler pipeline consisting of `LoadImaged`, `EnsureChannelFirstd`, `Orientationd`, `ScaleIntensityRangePercentilesd`, `Lambdad` (binary conversion), and `EnsureTyped`. No random cropping or data augmentation is applied to the validation data. The dataset is loaded using `CacheDataset` with a cache rate of 0.3, meaning that 30% of the data is cached in memory to accelerate data loading. We use a `DataLoader` with `batch_size=1`, `num_workers=2`, and `pin_memory=True`.

Inference

Because the full CT volume is much larger than the training patch size ($128 \times 128 \times 16$), it is not possible to feed the entire volume into the model at once. Therefore, we use the `sliding_window_inference` function provided by MONAI during inference. We set the region-of-interest size to `roi_size = (128, 128, 16)`, which is the same as the training patch size, and use `sw_batch_size = 2` to process two windows simultaneously. The overlapping areas between windows are combined by averaging the predictions using a Gaussian importance map, which helps produce smoother segmentation results. After obtaining the output logits from the model, we apply `argmax` to convert the predictions into the final binary segmentation mask, where 0 represents background and 1 represents the hemorrhage region.

Evaluation Metrics

We use seven metrics to evaluate the segmentation results. Table 3.3 shows the details:

Metric	Formula
Dice Score	$\frac{2TP}{2TP + FP + FN}$
IoU (Jaccard)	$\frac{TP}{TP + FP + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
Specificity	$\frac{TN}{TN + FP}$
HD95	Hausdorff Distance at 95th percentile (using <code>surface_distance</code> library)

Table 3.4: Evaluation metrics.

In our evaluation, Dice Score and IoU measure how much the predicted region overlaps with the ground truth. Precision shows how many of the predicted hemorrhage voxels are correct. Recall shows how much of the actual hemorrhage the model can detect. Specificity measures how well the model identifies the non-hemorrhage area. Accuracy is the overall correctness. HD95 measures the distance between the surfaces of the prediction and the ground truth at the 95th percentile, so a lower HD95 means the boundary is more accurate. We compute HD95 using the `surface_distance` library.

Chapter 4

EXPERIMENT AND RESULTS

4.1 ResNet50 Encoder

Training Results

We trained the model for 23 epochs before early stopping kicked in (out of a maximum of 50). The total training time was 41.3 minutes on a Tesla T4 GPU. The best validation Dice we got was 0.641 at epoch 15.

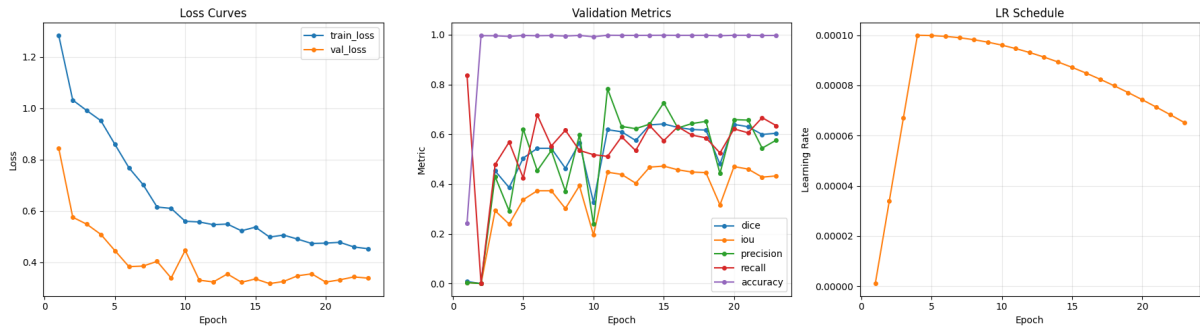


Figure 4.1: Training curves. Left: train and validation loss. Center: validation metrics over epochs. Right: learning rate schedule (warmup + cosine decay)

The loss curves: Both training loss and validation loss decreased steadily throughout the training process, going from 1.250 to 0.453 and from 1.095 to 0.338 respectively. We observed that the validation loss stayed below the training loss for most epochs. We believe this is because Dropout and heavy augmentation were only active during training, making the training samples more difficult. At validation time, these were turned off, so the model had an easier time with the clean data.

The validation metrics: The Dice score rose fast during the first 10 epochs, reaching around 0.51, but then it got stuck in the range of 0.58–0.64 for the rest of training. We also saw that precision and recall were quite unstable — in some epochs the model over-predicted (high recall, low precision), and in others it under-predicted (high precision, low recall). This suggests the model struggled to find a stable trade-off between detecting all hemorrhage and avoiding false alarms.

The learning rate schedule: During the warmup phase (epochs 1–3), the learning rate increased slowly from 1×10^{-6} to 1×10^{-4} . This is also when we saw the biggest jump

in Dice. After that, the cosine decay brought the learning rate down gradually, and by epoch 23 when training stopped, it was around 6.5×10^{-5} .

Test Set Results

We tested the model on all 50 test volumes that were kept completely separate during training. The table below shows the overall results:

Metric	Mean	Std	Median	Min	Max
Dice	0.444	± 0.323	0.549	0.0000	0.917
IoU	0.342	± 0.277	0.379	0.0000	0.847
Precision	0.500	± 0.337	0.638	0.0000	0.938
Recall	0.480	± 0.350	0.538	0.0000	0.978
HD95 (mm)	43.59	± 30.82	39.32	4.83	125.89

Table 4.1: Overall test results across 50 volumes.

The mean Dice score is 0.444, which is moderate. However, the standard deviation is very high (± 0.323), which means the results varied a lot between different volumes. The median Dice (0.549) is higher than the mean, which tells us that most volumes actually performed better than the average. The problem is that a few very bad cases (Dice close to 0) pulled the mean down significantly.

Visual Results

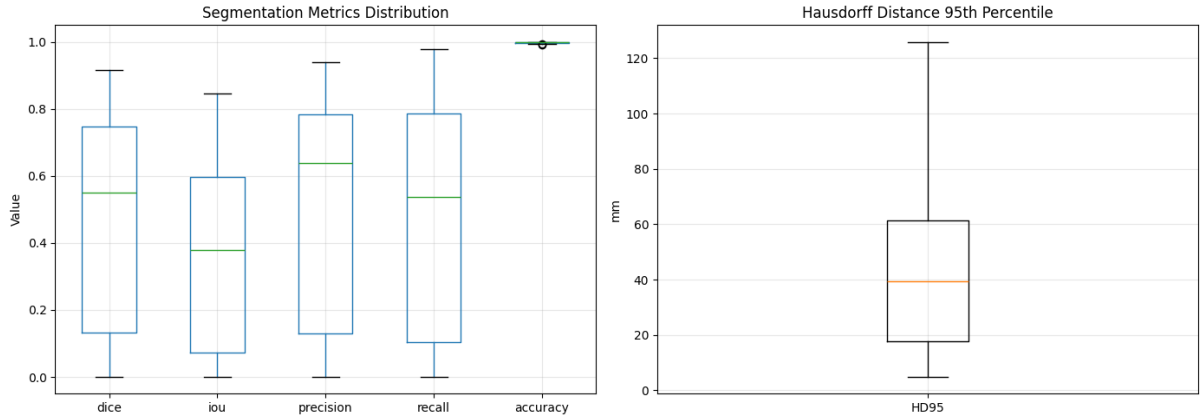


Figure 4.2: Box plots of test metrics (left) and HD95 distance (right) across 50 test volumes.

From the box plots, we can see several things. First, all metrics (Dice, IoU, precision, recall) have their lower whiskers reaching 0.0, which confirms that some volumes completely failed. Second, the boxes are quite tall, meaning there is a big difference between the best and worst results. Third, the accuracy box is squeezed near 1.0 - but this does not mean the model is good. Because hemorrhage only takes up a very small part of the brain (often less than 1%), even a model that predicts "no hemorrhage everywhere"

would get over 99% accuracy. So we should ignore accuracy and focus on Dice and IoU instead. Fourth, the HD95 box plot shows a median around 39 mm, but some volumes have values as high as 126 mm, which means the predicted boundaries were very far from the real ones in those cases.

Visual Results

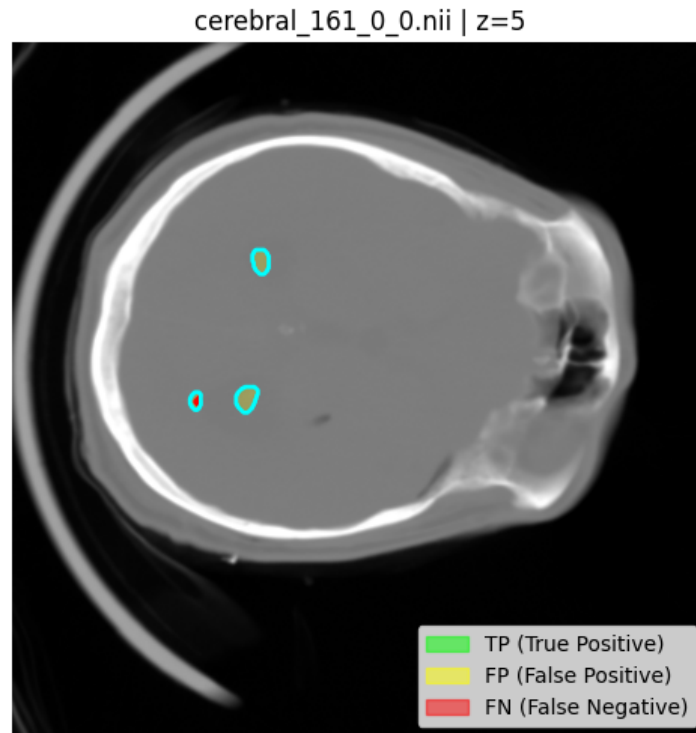


Figure 4.3: Prediction overlay on a single slice

This figure shows what the model predicts on a single CT slice. The cyan outlines mark the regions that the model thinks are hemorrhage. We can see that the predictions are in reasonable locations inside the brain. In this slice, there are no red areas, which means the model did not miss any hemorrhage here. However, we notice that the model produced several small separate regions instead of one connected area. This is a common problem with 2D slice-wise approaches because the model cannot see the neighboring slices. A simple post-processing step like connected component filtering could help remove these small false detections.

3D Mesh Visualization

3D Hemorrhage Mesh: cerebral_185_0_0.nii (Dice=0.9172)

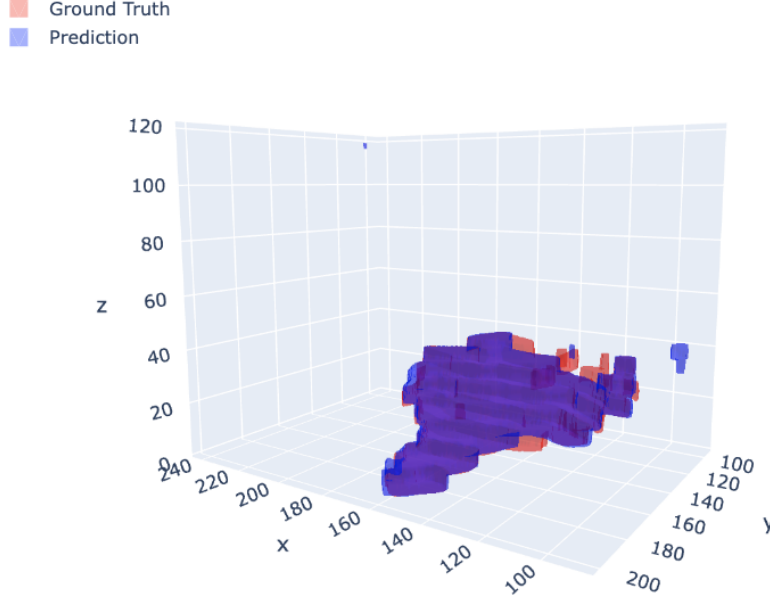


Figure 4.4: 3D mesh of the best case (cerebral_185, Dice = 0.917)

This 3D visualization shows how well the prediction matches the ground truth for the best case. The blue mesh (prediction) covers most of the red mesh (ground truth) very closely. We can see that the shape and size of the predicted hemorrhage match the real one well. There is a small blue fragment on the right side that seems to be a minor false positive, but it is very small compared to the main volume. This figure also shows that even though we trained the model on individual 2D slices, the predictions are consistent across slices and form a smooth 3D shape.

4.2 SegResNet

Training Results

We trained the SegResNet model for a maximum of 50 epochs. However, the training stopped early at epoch 47 because the Dice score on the validation set did not improve for 15 epochs in a row (early stopping with patience = 15). The best model was saved at epoch 32 with a validation Dice score of 0.3075.

Figure 4.5 shows the training curves. In the loss curves (left), we can see that both the training loss and validation loss decreased steadily from about 0.56 to around 0.40 and 0.44, respectively. The gap between training and validation loss is small, which means the model did not overfit too much. In the Dice and IoU curves (middle), we can see

that the model started to learn from epoch 4, and the scores kept increasing with some fluctuation. The Dice score reached its best value of 0.3075 at epoch 32. In the right chart, we can see that recall increased over time (the model detected more hemorrhage), while precision remained low, which means the model also produced many false positives.

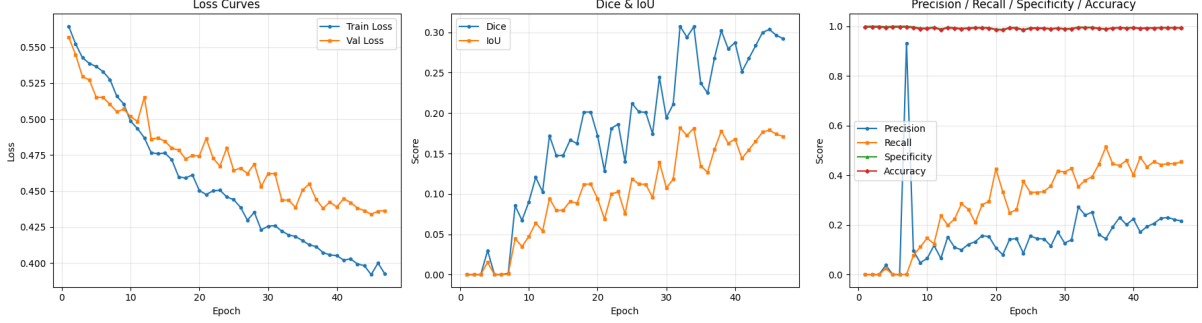


Figure 4.5: Training curves

Test Set Results

We evaluated the best model on the test set of 50 volumes. Table 4.2 shows the summary of all metrics:

Metric	Mean	Std	Median	Min	Max
Dice	0.1794	0.2058	0.0920	0.0000	0.6832
IoU	0.1148	0.1444	0.0482	0.0000	0.5188
Precision	0.1824	0.2283	0.0752	0.0000	0.7998
Recall	0.2210	0.2263	0.1564	0.0000	0.7236
Specificity	0.9968	0.0025	0.9979	0.9891	0.9997
HD95 (mm)	103.94	34.51	102.06	36.84	196.67

Table 4.2: Test set summary metrics.

The results show that the model achieved a mean Dice score of 0.1794 and a mean IoU of 0.1148 on the test set. The standard deviation is high (Dice std = 0.2058), which means the performance is very different across test cases. The best case reached a Dice of 0.6832, while many cases had a Dice of 0.0000 (the model could not detect any hemorrhage). The mean recall is 0.2210, which means the model can only detect about 22% of the hemorrhage voxels. The mean precision is 0.1824, which means many of the predicted voxels are false positives. The specificity (0.9968) and accuracy (0.9946) are very high because most voxels are background, so even a model that predicts nothing will have high accuracy. The mean HD95 is 103.94 mm, which is quite high and shows that the predicted boundaries are far from the ground truth in many cases.

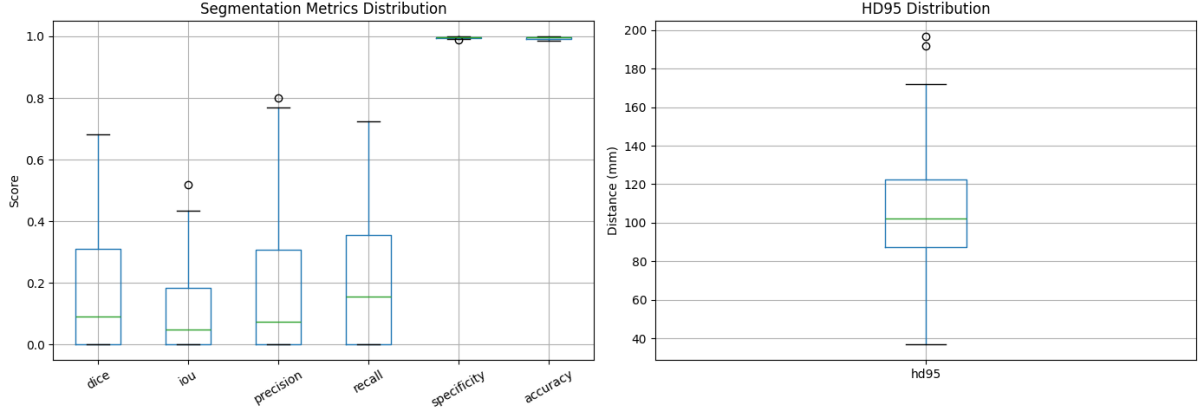


Figure 4.6: Distribution of segmentation metrics and HD95 on the test set.

Figure 4.6 shows the distribution of metrics across all 50 test cases. We can see that Dice, IoU, Precision, and Recall have very wide distributions with many values near 0. This means the model failed completely on some cases. Specificity and Accuracy are very high and close to 1.0 for all cases. The HD95 distribution shows most values are between 80 and 130 mm, with some outliers reaching nearly 200 mm.

Visual Results

Figure 4.7 shows a visual comparison between the ground truth and the model prediction on a test case (cerebral_161_0_0.nii, slice z=4). The left image shows the CT scan with ground truth in red. The middle image shows the prediction in blue. The right image shows the TP (True Positive, green), FP (False Positive, yellow), and FN (False Negative, red/pink) overlay.

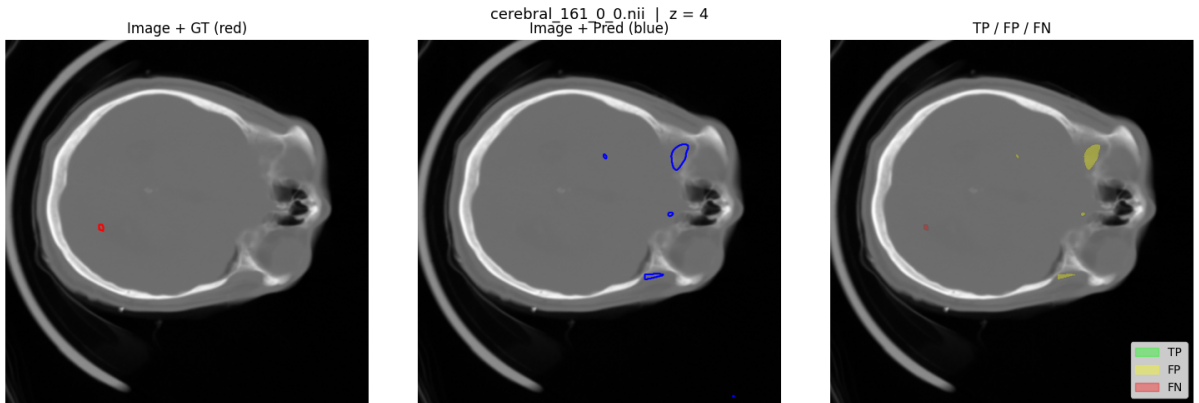


Figure 4.7: Visual comparison on test case cerebral_161_0_0.nii (z=4)

From Figure 4.7, we can see that the model predicted some regions that do not match the ground truth (yellow areas = FP). The ground truth hemorrhage (red in left image) was very small, and the model missed it (pink area = FN). The model also predicted hemorrhage in wrong locations. This visual result is consistent with the low Dice and Precision scores in Table 4.2.

3D Mesh Visualization

To better understand the segmentation result in 3D, we created a 3D mesh visualization for the best test case (cerebral_189_0_0.nii, Dice = 0.6832). We used the marching cubes algorithm from the scikit-image library to extract the surface mesh from the binary masks. Figure 4.2.4 shows the 3D mesh of the ground truth (red) and the prediction (blue).

3D Hemorrhage Mesh: cerebral_189_0_0.nii

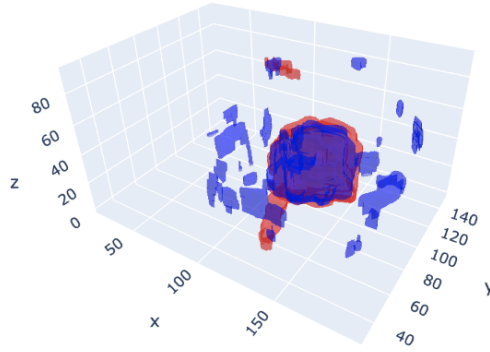


Figure 4.8: 3D hemorrhage mesh on cerebral_189_0_0.nii.

From Figure 4.8, we can see that the predicted region (blue) covers most of the ground truth region (red) in the center, which means the model can detect the main hemorrhage area. However, there are many small blue regions scattered around the brain that do not match the ground truth. These are false positive predictions. The red areas that are not covered by blue are false negatives, where the model missed the hemorrhage. This visualization confirms that the model has learned to find the general location of hemorrhage but still struggles with accurate boundary detection and produces extra false predictions.

4.3 Discussion

We used two models for brain hemorrhage segmentation: SegResNet (3D) and ResNet50 - UNet (2D). We tested both models on the same 50 test volumes. Table 4.3 shows the results.

Metric	SegResNet (3D)	ResNet50-UNet (2D)
Dice (mean)	0.1794	0.4439
IoU (mean)	0.1148	0.3421
Precision (mean)	0.1824	0.5004
Recall (mean)	0.2210	0.4799
Accuracy (mean)	0.9946	0.9981
HD95 (mean, mm)	103.94	43.59
Best Dice	0.6832	0.9172
Best HD95 (mm)	36.84	4.83
Parameters	18,796,002	32,521,105
Best val Dice	0.3075	0.6413
Stopped at	Epoch 47	Epoch 23

Table 4.3: Test set results of SegResNet and ResNet50-UNet.

From Table 4.3, we can see that ResNet50-UNet is better than SegResNet on all metrics. The Dice score is about 2.5 times higher (0.4439 vs 0.1794). The precision went up from 0.18 to 0.50, so ResNet50-UNet made fewer wrong predictions. The recall also improved from 0.22 to 0.48, so it can find more hemorrhage areas. The HD95 dropped from 103.94 mm to 43.59 mm, which means the prediction boundary is much closer to the real boundary.

Chapter 5

CONCLUSION

Conclusion

This project studied the problem of automatic brain hemorrhage segmentation from CT scans using deep learning methods. The dataset consisted of 3D medical images stored in the NIfTI format together with corresponding hemorrhage masks. Two different approaches were explored in this study: a 2D segmentation model based on ResNet50–UNet and a 3D segmentation model based on SegResNet. Experimental results show that the 2D ResNet50–UNet model achieved better performance than the 3D SegResNet model on the test dataset. The ResNet50–UNet obtained a mean Dice score of 0.4439, while SegResNet reached 0.1794. Other metrics such as IoU, precision, recall, and HD95 also show a consistent advantage for the 2D model.

However, both models still show large variations in performance across different volumes. The high standard deviation in Dice scores indicates that the models work well on some cases but fail completely on others. One important reason is that hemorrhage regions often occupy a very small part of the brain volume, which creates a severe class imbalance problem.

Future Works

For future work, several improvements could be explored. Using a larger and more diverse dataset may help the model generalize better. Additional data augmentation strategies and more advanced architectures could also improve performance. Post-processing techniques, such as connected component filtering or 3D consistency constraints, may help reduce false positives and produce cleaner segmentation results.

Overall, this project demonstrates the potential of deep learning methods for medical image segmentation and highlights both the opportunities and challenges in automatic brain hemorrhage detection.

References

- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234–241.
- Cardoso, M. J., Li, W., Brown, R., et al. (2022). MONAI: An open-source framework for deep learning in healthcare imaging. *Nature Methods*, 19, 1166–1167.
- Myronenko, A. (2018). 3D MRI Brain Tumor Segmentation Using Autoencoder Regularization. *International MICCAI Brainlesion Workshop*, 311–320.
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal Loss for Dense Object Detection. *IEEE International Conference on Computer Vision (ICCV)*, 2980–2988.
- Milletari, F., Navab, N., & Ahmadi, S. (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. *International Conference on 3D Vision (3DV)*, 565–571.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems (NeurIPS)*.